

AI-GenBench Benchmark Analysis

Architecture Survey, Dataset Contamination Assessment, and Submission Strategy

Project: DeepFakeDetector — McMaster AI Society

Issue: [#65](#)

Authors: DeepFakeDetector Research Team

Date: 2026-02-24

External Resources:

- Paper: [arXiv:2504.20865](#) — Pellegrini et al., IJCNN Verimedia 2025
 - Second paper: [arXiv:2511.21507](#) — "Generalized Design Choices for Deepfake Detectors"
 - GitHub: <https://github.com/MI-BioLab/AI-GenBench>
-

Table of Contents

1. [Abstract](#)
2. [Introduction](#)
3. [AI-GenBench Benchmark Overview](#)
4. 2.1 [Dataset Composition](#)
5. 2.2 [Temporal Evaluation Framework](#)
6. 2.3 [Evaluation Metrics](#)
7. [Architecture Analysis of Top-Performing Models](#)
8. 3.1 [Benchmark Results](#)
9. 3.2 [Architecture Deep Dive](#)
10. 3.3 [Key Design Choices](#)
11. 3.4 [Comparison Against Our Current Models](#)
12. 3.5 [Actionable Recommendations](#)
13. [Dataset Overlap Analysis](#)
14. 4.1 [AI-GenBench Source Datasets](#)
15. 4.2 [Our Training Data Sources](#)
16. 4.3 [Overlap Matrix](#)
17. 4.4 [Contamination Risk Assessment](#)
18. 4.5 [Impact on Benchmark Validity](#)
19. 4.6 [Clean Data Strategy](#)

- 20. [Testing Strategy and Submission Process](#)
- 21. 5.1 [Accessing AIGenBench Data](#)
- 22. 5.2 [Expected I/O Format and Training Protocol](#)
- 23. 5.3 [Evaluation Metrics and Scoring](#)
- 24. 5.4 [Submission Requirements](#)
- 25. 5.5 [Adaptation Plan for DeepFakeDetector](#)
- 26. [Recommendations and Roadmap](#)
- 27. [References](#)

Abstract

AI-GenBench is a 2025 IJCNN benchmark for AI-generated image detection, uniquely distinguished by its **temporal evaluation framework**: 36 generative models spanning 2017–2024 (from CycleGAN to FLUX 1) are ordered chronologically, and detectors are evaluated on their ability to generalize to **unseen, newer generators** after incremental training on older ones. The dataset contains 360K images (180K real + 180K fake) balanced across all 36 generators, sourced from ImageNet, COCO2017, LAION-400M, and RAISE for real content.

Architecture findings: The best-performing model on the AIGenBench benchmark is **ViT-L/14 pretrained with DINOv2** (fine-tuned), achieving **94.24% AUROC** on the "Next Period" generalization evaluation. ViT-L/14 CLIP (fine-tuned) reaches 92.04%, while ResNet-50 CLIP lags at 81.77%. Full fine-tuning consistently beats linear probing, and resizing beats multi-crop for generalization to novel generators.

Critical contamination finding: Our current training pipeline has direct dataset overlap with AIGenBench test sets in **7 of 9 evaluation windows**. Specifically: (1) DRAGON consolidates Synthbuster and GenImage, both of which are AIGenBench source datasets; (2) OpenFake contains SD 1.5/2.1/XL, DALL-E 3, and FLUX — generators covered in AIGenBench windows w7–w8; (3) WildFake's GAN content overlaps with AIGenBench's ForenSynths-sourced windows w0–w3. Any AIGenBench leaderboard submission using current training data would yield **inflated and invalid scores**.

Recommendations: Upgrade our ViT backbone to ViT-L/14 DINOv2; retrain exclusively on AIGenBench's own clean training split for benchmark submission; adopt JPEG compression and blur augmentation matching AIGenBench's DetermAugment protocol; and retain our unique FFT and gradient-field modules as potential competitive advantages in frequency-domain generalization.

1. Introduction

The central challenge in AI-generated image detection is not in-distribution performance — classifiers trained and tested on the same generator distribution routinely achieve >99% accuracy — but **out-of-distribution generalization**: can a detector trained on older methods still catch images from generators it has never seen?

This is exactly the question AI-GenBench is designed to answer. Unlike static benchmarks that partition images from the same generator pool into train/test splits, AI-GenBench enforces a **temporal train-then-test protocol**: a model trained on generators from 2017–2020 must detect images from 2021–2022 generators it was never exposed to, and so on across 9 chronological cohorts. This mirrors the real-world adversarial scenario where new generation methods outpace deployed detectors.

For the DeepFakeDetector project, this benchmark is especially relevant because:

1. Our fusion model (CNN + ViT + Gradient Field) is specifically designed for generalization across generation methods.
2. Our training data (OpenFake, DRAGON, WildFake) spans multiple generation eras, raising contamination concerns for fair evaluation.
3. The benchmark provides a standardized leaderboard for comparing our hybrid approach against the state of the art.

This document answers the three research questions posed in [issue #65](#):

1. What architectures perform best on AIGenBench, and what design choices drive that performance?
2. Is there dataset overlap between our training data and AIGenBench's test generators?
3. How do we properly access, evaluate against, and submit to AIGenBench?

2. AIGenBench Benchmark Overview

2.1 Dataset Composition

AI-GenBench contains images from **36 generative AI models** released between 2017 and 2024, covering the full arc from early-era GANs to state-of-the-art diffusion models.

Dataset statistics:

Split	Real Images	Fake Images	Total	Per Generator
Train	144,000	144,000	288,000	4,000
Eval	36,000	36,000	72,000	1,000
Total	180,000	180,000	360,000	5,000

Real images are sourced from four established datasets to ensure diverse, high-quality, unmanipulated content:

- **ImageNet** (ILSVRC2012) — 1,000-class object recognition images
- **COCO 2017** — natural scene images with diverse content
- **LAION-400M** — large-scale web-crawled images
- **RAISE** — RAW-format uncompressed photography

Table 1: All 36 Generators Organized by Sliding Window

Window	Generator 1	Generator 2	Generator 3	Generator 4	Era
w0	CycleGAN	Cascaded Refinement Nets	ProGAN	StarGAN	2017–2018
w1	SN-PatchGAN	BigGAN	IMLE	StyleGAN1	2018–2019
w2	GauGAN	StyleGAN2	DDPM	CIPS	2019–2020
w3	VQGAN	GANformer	ADM	StyleGAN3	2020–2021
w4	LaMa	FaceSynthetics	ProjectedGAN	Palette	2021–2022
w5	VQ-Diffusion	Denoising Diffusion GAN	Glide	Latent Diffusion	2021–2022
w6	Midjourney	MAT	DiffusionGAN (ProjGAN)	DiffusionGAN (SG2)	2022
w7	Stable Diffusion 1.4	Stable Diffusion 1.5	Stable Diffusion 2.1	DeepFloyd IF	2022–2023
w8	SDXL 1.0	DALL-E 3	FLUX 1 Dev	FLUX 1 Schnell	2023–2024

This spans the full generative AI history: early image-to-image translation GANs (w0), large-scale class-conditional GANs (w1–w2), the diffusion model emergence (w3–w5), and modern text-to-image systems (w6–w8).

Source repositories assembled into AIGenBench:

Source Dataset	Generators Covered
ForenSynths	ProGAN, CycleGAN, StarGAN, GauGAN, BigGAN, CRN, IMLE, SN-PatchGAN, ProjectedGAN, StyleGAN1/2
GenImage	ADM, BigGAN, GLIDE, VQGAN, VQDM, SD variants
Synthbuster	DALL-E 2/3, Adobe Firefly, Midjourney v5, SD 1.3/1.4/2, GLIDE

Source Dataset	Generators Covered
ELSA-D3	SD 1.4, SD 2.1, SDXL 1.0, DeepFloyd IF
DDMD	Denoising diffusion detection (early diffusion era)
DMID	DM Image Detection (class-conditional, noise-to-image, text-to-image)
DRCT	Diffusion-based reconstruction / generation
ImagiNet	General-purpose generation benchmark
SFHQ-T2I	Text-to-image face generation
Aeroblade	Diffusion model detection artifacts
PolarDiffShield	Adversarially-generated images

2.2 Temporal Evaluation Framework

The core innovation of AI-GenBench is the **sliding window evaluation protocol** that prevents overfitting to a static generator distribution.

Algorithm 1 — AIGenBench Temporal Training & Evaluation Protocol:

```

Input:  ts = 9  (number of time steps)
        am = 4  (augmentation multiplier)
        n  = 1  (training epochs per step)

For training step k = 0, 1, ..., 8:

    [TRAINING]
    1. If k = 0: initialize model  $\theta_0$  randomly
       Else: reload weights  $\theta_{k-1}$  from previous step
    2. Collect training data: all generators in windows  $w_0, w_1, \dots, w_k$ 
       (4,000 images per generator, 144,000 real images total)
    3. Apply CustomAugment with multiplier am=4:
       → Each image produces 4 augmented versions per epoch
    4. Train for n=1 epoch on the collected training data

    [EVALUATION]
    5. Apply DetermAugment (deterministic, multiplier=1) to eval set
    6. Report AUROC on:
       - "Next Period": eval generators =  $w_{k+1}$           ← generalization
       - "Past Period": eval generators =  $w_0, \dots, w_k$     ← retention
       - "Whole Period": eval generators =  $w_0, \dots, w_{k+1}$ 

Output: Per-step AUROC matrix, mean Next-Period AUROC ( $\bar{G}$ )

```

Visual representation of the sliding window protocol:

Training	Data Seen	Test Window (Next Period)

Step 0	[w0]	→	w1 (StyleGAN, BigGAN...)
Step 1	[w0, w1]	→	w2 (StyleGAN2, DDPM...)
Step 2	[w0, w1, w2]	→	w3 (VQGAN, ADM...)
Step 3	[w0, w1, w2, w3]	→	w4 (LaMa, Palette...)
Step 4	[w0 ... w4]	→	w5 (Glide, LatentDiff...)
Step 5	[w0 ... w5]	→	w6 (Midjourney, MAT...)
Step 6	[w0 ... w6]	→	w7 (SD 1.4, SD 1.5...) ← major difficulty
jump			
Step 7	[w0 ... w7]	→	w8 (SDXL, DALL-E 3...) ← hardest

The critical difficulty jump occurs at Step 6 → w7: the transition from GAN-era models to Stable Diffusion represents a qualitative shift in generation artifacts. The benchmark paper documents a notable AUROC drop at this step for all evaluated methods.

2.3 Evaluation Metrics

Primary Metric: AUROC

The Area Under the Receiver Operating Characteristic Curve measures the probability that a randomly chosen fake image receives a higher anomaly score than a randomly chosen real image:

$$\text{AUROC} = \int_0^1 \text{TPR}(\text{FPR}) \, d(\text{FPR}) = P(\hat{p}_{\text{fake}}(x^+) > \hat{p}_{\text{fake}}(x^-) | \text{right})$$

where x^+ denotes a fake sample, x^- a real sample, and $\hat{p}_{\text{fake}}$ is the model's predicted probability of being fake. AUROC = 1.0 is perfect; AUROC = 0.5 is random chance.

AUROC is preferred over accuracy in this benchmark because the decision threshold calibration varies across generator distributions — AUROC is threshold-free.

Temporal Generalization Score

We define the **mean temporal generalization AUROC** $\bar{G}$ as the primary aggregate metric across all sliding window steps:

$$\bar{G} = \frac{1}{|\mathcal{T}| - 1} \sum_{k=1}^{|\mathcal{T}|-1} \text{AUROC}(\theta_k; \mathcal{D}\{w_{k+1}\})$$

where:

- $|\mathcal{T}| = 9$ (9 time steps)
- θ_k = model weights after training on windows $w_0, \dots, w_k$
- $\mathcal{D}\{w_{k+1}\}$ = evaluation set for the next unseen window

This is averaged over 8 "next-period" evaluations (steps 1-8), with $k=0$ excluded since no prior context exists.

Supporting Metrics

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \quad \text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

3. Architecture Analysis of Top-Performing Models

3.1 Benchmark Results

The following results are drawn directly from the AIGenBench paper (arXiv:2504.20865, Table III) and report mean AUROC and accuracy on the **Next Period** evaluation, averaged across all 8 sliding window steps.

Table 2: Baseline Method Results on AIGenBench "Next Period" (Mean across 8 steps)

Architecture	Backbone Params	Training Mode	Image Adaptation	AUROC	Accuracy
ResNet-50 CLIP	~25M	Fine-tune	Resize	81.77%	73.42%
ResNet-50 CLIP	~25M	Fine-tune	Multi-crop	76.51%	67.22%
ResNet-50 CLIP	~25M	Linear probe	Resize	77.66%	69.85%
ResNet-50 CLIP	~25M	Linear probe	Multi-crop	69.08%	63.33%
ViT-L/14 CLIP	~307M	Fine-tune	Resize	92.04%	85.28%
ViT-L/14 CLIP	~307M	Fine-tune	Multi-crop	87.66%	76.83%
ViT-L/14 CLIP	~307M	Linear probe	Resize	88.47%	76.25%
ViT-L/14 CLIP	~307M	Linear probe	Multi-crop	68.74%	63.02%
ViT-L/14 DINOv2	~307M	Fine-tune	Resize	94.24%	84.09%
ViT-L/14 DINOv2	~307M	Fine-tune	Multi-crop	93.44%	78.64%
ViT-L/14 DINOv2	~307M	Linear probe	Resize	88.47%	79.34%
ViT-L/14 DINOv2	~307M	Linear probe	Multi-crop	88.74%	78.09%

Table 3: Training Times (Intel i9-10900X + NVIDIA RTX 3080 Ti)

Architecture	Training Mode	Total Time (9 steps)
ResNet-50 CLIP	Fine-tune	4.87 h
ResNet-50 CLIP	Linear probe	4.87 h

Architecture	Training Mode	Total Time (9 steps)
ViT-L/14 CLIP	Fine-tune	19.87 h
ViT-L/14 CLIP	Linear probe	6.35 h
ViT-L/14 DINOv2	Fine-tune	26.00 h
ViT-L/14 DINOv2	Linear probe	7.50 h

Key observations:

1. ViT-L/14 DINOv2 (fine-tuned, resize) achieves the highest AUROC at **94.24%** — 2.2 points above the second-best.
2. Larger models consistently generalize better (307M > 25M).
3. Full fine-tuning consistently outperforms linear probing on AUROC (despite being more expensive).
4. Resize consistently outperforms multi-crop — multi-crop introduces random spatial contexts that hurt generalization to unseen generators.
5. Notable performance cliff at window transitions involving the Stable Diffusion family (w6 → w7).

3.2 Architecture Deep Dive

3.2.1 ResNet-50 CLIP (~25M Parameters)

ResNet-50 is a 50-layer convolutional network with 4 residual stages, pretrained via CLIP's contrastive image-text objective. Each residual block computes:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s \mathbf{x}$$

where $\mathcal{F}$ represents the stacked convolutions (BN → ReLU → Conv sequence) and W_s is a shortcut projection. Global Average Pooling over the final feature map produces a 1024-D embedding:

$$\mathbf{z} = \text{GAP}(\mathbf{F}_{\text{last}}) \in \mathbb{R}^{1024}$$

Why it under-performs on AIGenBench: ResNet's local receptive fields and fixed convolutional inductive bias capture pixel-level spatial artifacts well but miss long-range semantic inconsistencies. Modern diffusion models (SD, DALL-E 3, FLUX) produce images with locally convincing textures but globally incoherent structure — precisely what a convolutional network without global attention fails to detect.

CLIP pretraining provides some generalization benefit by aligning visual features with semantic concepts, but the ResNet backbone's architectural limitations dominate.

3.2.2 ViT-L/14 CLIP (~307M Parameters)

Vision Transformer Large with 14×14 patch size. An image of size $H \times W$ is divided into $N = (H/14) \times (W/14)$ non-overlapping patches. Each patch is linearly projected to a d -dimensional embedding:

$$\mathbf{z}_0 = [\mathbf{x}_{\text{cls}}; \mathbf{x}_1^{\text{p}}; \mathbf{x}_2^{\text{p}}; \dots; \mathbf{x}_N^{\text{p}}] + \mathbf{E}_{\text{pos}}$$

where $\mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times d}$ is the patch projection matrix and $\mathbf{E}_{\text{pos}}$ is learned positional encoding.

Each of the 24 transformer layers applies **multi-head self-attention (MSA)**:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\text{top}}}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, $h=16$ heads, $d_k = d/h = 64$.

The classification token $\mathbf{z}_L$ (output of the final layer) is fed to a linear classification head. For AIGenBench, this head is replaced with a binary fake/real head during fine-tuning.

CLIP pretraining aligns image features with natural language descriptions across 400M image-text pairs. This cross-modal semantic grounding helps generalization: the model learns to identify what an image depicts beyond low-level pixel statistics, making it robust to generator-specific frequency artifacts that shift across generation eras.

3.2.3 ViT-L/14 DINOv2 (~307M Parameters) — Best Performer

Same ViT-L/14 architecture, but pretrained via **self-supervised knowledge distillation with no labels (DINO)**. DINOv2 uses a teacher-student framework where both teacher and student share the same architecture, but the teacher's weights are updated via an exponential moving average (EMA) of the student:

$$\theta_t \leftarrow \lambda \theta_t + (1 - \lambda) \theta_s, \quad \lambda \approx 0.9996$$

The student is trained to match the teacher's output distribution using a **cross-entropy self-distillation loss**:

$$\mathcal{L}_{\text{DINO}} = -\sum_x p_t(x) \log p_s(x)$$

where $p_t(x) = \text{softmax}\left(\frac{g_{\theta_t}(x) - c}{\tau_t}\right)$ and $p_s(x) = \text{softmax}\left(\frac{g_{\theta_s}(x)}{\tau_s}\right)$, with centering offset c and separate temperature hyperparameters $\tau_t < \tau_s$ (teacher is sharper). Centering prevents mode collapse; temperature differential creates a training signal.

DINOv2 also adds a patch-level self-supervised objective (iBOT) where random patches are masked and reconstructed:

$$\mathcal{L}_{\text{iBOT}} = -\sum_{i \in \mathcal{M}} p_t(\tilde{\mathbf{z}}_i) \log p_s(\hat{\mathbf{z}}_i)$$

where $\mathcal{M}$ is the set of masked patches, $\hat{\mathbf{z}}_i$ are the masked student tokens, and $\tilde{\mathbf{z}}_i$ are the corresponding unmasked teacher tokens.

Why DINOv2 excels over CLIP for deepfake detection:

Property	CLIP	DINOv2
Pretraining objective	Image-text contrastive	Self-supervised image distillation
Supervision signal	Weak text annotations	No annotations
Feature type	Semantically-aligned with language	Dense spatial + semantic
Attention head specialization	Semantic regions	Fine-grained texture + structure
Patch-level features	Weak	Strong (iBOT objective)
Sensitivity to local artifacts	Low	High

DINOv2's dense patch-level objectives force the model to attend to fine-grained texture patterns — exactly the signal that distinguishes real from AI-generated images at the forensic level. CLIP's image-level contrastive objective averages these details away.

3.3 Key Design Choices (from arXiv:2511.21507)

The companion paper "Generalized Design Choices for Deepfake Detectors" systematically isolates factors that drive AIGenBench performance:

A. Augmentation Strategy (Most Impactful Factor)

The DetermAugment pipeline (used at evaluation time) applies a cascade of real-world degradations. Training with matching augmentations dramatically improves generalization:

Augmentation	Parameter Range	Effect
JPEG compression	quality $\in$ [50, 99]	Simulates social media re-compression; destroys subtle frequency artifacts
Gaussian blur	$\sigma \in$ [0, 2]	Simulates camera focus blur and anti-aliasing
Gaussian noise	$\sigma \in$ [0, 0.01]	Simulates sensor noise; regularizes texture-level features
Resize	scale $\in$ [0.5, 2.0]	Simulates reposting at different resolutions
Center crop	224×224	Standardizes spatial extent

Training without JPEG compression simulation causes a significant drop in AUROC on w7-w8 (SD/FLUX era) because diffusion models already incorporate lossy compression characteristics.

B. Input Resolution Strategy

Strategy	Description	Generalization
	Uniformly downsample/upsample to 224×224	Better (consistent spatial statistics)

Strategy	Description	Generalization
Resize to 224		
Multi-crop	Extract multiple 224×224 crops from large image	Worse (inconsistent spatial context)

Multi-crop's apparent benefit on training distribution is misleading: it creates artificially diverse spatial views but inconsistent global context, which hurts when generalizing to generators with different spatial consistency patterns.

C. Training Regime

- **Full fine-tuning > linear probe** by 3–5 AUROC points across all architectures. Frozen backbone representations are not optimally calibrated for forensic artifact detection — fine-tuning allows the model to de-emphasize semantic features and amplify artifact-indicative channels.
- **Low learning rate for fine-tuning:** The paper uses a cosine-decay schedule with warm-up starting from $\eta = 5 \times 10^{-5}$. Aggressive learning rates destroy pretrained representations.

D. Multiclass Generator Training

Training with **generator-specific labels** (36-class classification, one class per generator) rather than binary real/fake improves binary AUROC at test time:

$$\mathcal{L}_{\text{multi}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c \in \mathcal{C}} \mathbb{1}[y_i = c] \log p(c \mid x_i)$$

At inference, the binary fake probability is computed as $p(\text{fake} \mid x) = 1 - p(\text{real} \mid x)$.

Multiclass training forces the model to learn **generator-discriminative features** rather than a coarse real/fake boundary. These discriminative features generalize better to unseen generators that share attributes with seen ones (e.g., a new diffusion model shares characteristics with older diffusion models).

E. Incremental Update Strategy

At each training step $k > 0$, the model reloads θ_{k-1} rather than reinitializing. Reinitializing at each step causes **catastrophic forgetting** — the model loses its ability to detect older generator types.

The paper evaluates several continual learning strategies. The simplest — just reloading from the previous step — is surprisingly competitive with more complex approaches (replay buffers, elastic weight consolidation), suggesting that the training data expansion at each step (more generators → more samples) provides implicit replay.

3.4 Comparison Against Our Current Models

Table 4: DeepFakeDetector Models vs AIGenBench Top Performers

Model	Architecture	Params	Pretraining Strategy	In-Dist AUROC	Est. AIGenBench $\bar{G}$	Gap to SOTA
CNN Transfer	EfficientNet-B0	5.3M	ImageNet supervised	~90%	~65-72% (est.)	~22 pts
ViT-Base	ViT-B/16	86M	ImageNet-21k supervised	~88%	~78-82% (est.)	~12 pts
DeiT-Small	DeiT-S/16	22M	ImageNet distilled	~85%	~75-79% (est.)	~15 pts
FFT CNN	CompactFFTNet	0.5M	Scratch	~82%	~60-65% (est.)	~30 pts
Gradient Field	CompactGradientNet	1.0M	Scratch	~84%	~62-67% (est.)	~28 pts
Fusion LogReg	4-D probability stack	5 params	N/A	~91%	~75-80% (est.)	~15 pts
AIGenBench SOTA	ViT-L/14 DINOv2	307M	Self-supervised (DINO)	N/A	~94.24%	— Target

In-distribution AUROC is reported on our own train/test splits and should not be directly compared to AIGenBench $\bar{G}$.

Key gap analysis:

- Scale gap:** Our largest model (ViT-Base, 86M) has ~3.6× fewer parameters than ViT-L/14 (307M). Empirically, scale is the strongest predictor of generalization in transformer-based detectors.
- Pretraining gap:** Our models use supervised ImageNet pretraining. DINOv2's self-supervised patch-level objective produces more forensically-sensitive features. CLIP's contrastive objective is also stronger than ImageNet classification for generalization.
- Feature coverage:** Our FFT and gradient-field modules capture frequency and edge-coherence artifacts that the ViT-L baseline does not include. This is a **potential advantage** — no top AIGenBench model uses frequency-domain features, and they may provide complementary signal for FLUX/SDXL-era generators.
- Augmentation gap:** Our current training pipeline does not include systematic JPEG compression simulation with quality $\in [50, 99]$. This is the single most impactful fix.

3.5 Actionable Recommendations for DeepFakeDetector

Priority 1 (Architecture):

- Replace ViT-Base backbone with **ViT-L/14 pretrained with DINOv2** (via `torch.hub` or HuggingFace)
- Keep EfficientNet-B0 as a lightweight ensemble member

Priority 2 (Training strategy):

- Switch to **full fine-tuning** with learning rate $\eta = 5 \times 10^{-5}$, cosine decay, warm-up of 500 steps
- Add **JPEG compression simulation** to training augmentations:
`torchvision.transforms.functional.jpeg` with quality uniformly sampled from [50, 99]
- Add Gaussian blur: `GaussianBlur(kernel_size=5, sigma=(0.1, 2.0))` with $p=0.5$

Priority 3 (Training objective):

- Experiment with **multiclass training** (36 generator classes → binary at inference) on the AIGenBench training split

Priority 4 (Evaluation):

- Integrate AIGenBench's `evaluate_all_windows.py` into our evaluation pipeline for honest comparison
- Run existing ViT-Base model through AIGenBench protocol (clean data only) to establish a true baseline before upgrading

Priority 5 (Ensemble):

- Explore fusion of ViT-L/14 DINOv2 + FFT CNN — no published AIGenBench submission uses frequency-domain features, which may provide complementary signal on SD/FLUX-era generators

4. Dataset Overlap Analysis

4.1 AIGenBench Source Datasets

AIGenBench images are drawn from the following source collections (from `ai_gen_bench_generators.py` and `DATASET_STATS.md`):

Source Dataset	Generators Covered	Windows
ForenSynths	CycleGAN, CRN, ProGAN, StarGAN, SN-PatchGAN, BigGAN, IMLE, StyleGAN1, GauGAN, StyleGAN2, ProjectedGAN	w0–w2, w4
DDMD	DDPM, Denoising Diffusion GAN	w2, w5
DMID	CIPS, VQGAN, GANformer, VQ-Diffusion, Latent Diffusion, Palette	w2–w5
GenImage	ADM, BigGAN, GLIDE, VQDM, StyleGAN3, SD variants	w3, w5

Source Dataset	Generators Covered	Windows
Synthbuster	DALL-E 2/3, Adobe Firefly, Midjourney v5, SD 1.3/1.4/2, GLIDE	w5-w8
ELSA-D3	SD 1.4, SD 2.1, SDXL 1.0, DeepFloyd IF	w7-w8
DRCT	DiffusionGAN (ProjectedGAN/StyleGAN2 variants)	w6
ImagiNet	FaceSynthetics, ProjectedGAN, LaMa, MAT, Midjourney, SD, SD XL, DALL-E 3	w4, w6-w8
SFHQ-T2I	Face-specific text-to-image (SD-based)	w7
Aeroblade	Latent Diffusion, SDXL, DeepFloyd IF	w5, w7-w8
PolarDiffShield	Adversarially-generated content (SD-based)	w7-w8

4.2 Our Training Data Sources

Our Dataset	Image Count	Generators / Sources	HuggingFace / Source
OpenFake	4M+	SD 1.5/2.1/XL/3.5, Flux Dev/Schnell/1.1-Pro, Midjourney v6/v7, DALL-E 3, Imagen 3/4, GPT Image 1, Ideogram 3.0, Grok-2, Recraft v3, HiDream-I1	ComplexDataLab/OpenFake
DRAGON	2.5M	Consolidates: CiFAKE + Diffusion Forensics + Synthbuster + GenImage	lesc-unifi/dragon
WildFake	3.57M	GANs (ProGAN, StyleGAN 1/2, BigGAN, ADM, Midjourney v5)	ModelScope: hy2628982280/WildFake
Manual-gen-images	100s	DALL-E, Midjourney, SD variants	DeepFakeDetector/manual-gen-images
Open-Images-V7	30+ (sampled)	Real images only	bitmind/open-images-v7

4.3 Overlap Matrix

We measure dataset-level overlap using the **Jaccard similarity coefficient**:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

where A is the set of generators in our training data and B is the set of generators in a given AIGenBench evaluation window.

Table 5: Contamination Risk by AIGenBench Window

AI GenBench Window	Generators	AI GenBench Source	Contaminating Our Dataset	Risk
w0 (2017–18)	CycleGAN, CRN, ProGAN, StarGAN	ForenSynths	WildFake (ProGAN, StyleGAN-family); DRAGON→GenImage (ProGAN via ForenSynths)	HIGH
w1 (2018–19)	SN-PatchGAN, BigGAN, IMLE, StyleGAN1	ForenSynths	WildFake (BigGAN, StyleGAN); DRAGON→GenImage (BigGAN)	HIGH
w2 (2019–20)	GauGAN, StyleGAN2, DDPM, CIPS	ForenSynths, DDMD	WildFake (StyleGAN2)	MEDIUM
w3 (2020–21)	VQGAN, GANformer, ADM, StyleGAN3	GenImage, DMID	DRAGON→GenImage (ADM, VQGAN)	HIGH
w4 (2021–22)	LaMa, FaceSynthetics, ProjectedGAN, Palette	ForenSynths, DMID, ImagiNet	Limited overlap	LOW
w5 (2021–22)	VQ-Diffusion, DDG, Glide, Latent Diffusion	GenImage, Synthbuster, DMID	DRAGON→GenImage (Glide); DRAGON→Synthbuster (Glide, SD 1.4)	HIGH
w6 (2022)	Midjourney, MAT, DiffGAN×2	Synthbuster, ImagiNet	DRAGON→Synthbuster (Midjourney v5); OpenFake (Midjourney v6/v7)	HIGH
w7 (2022–23)	SD 1.4, SD 1.5, SD 2.1, DeepFloyd IF	ELSA-D3, Synthbuster, SFHQ-T2I	DRAGON→Synthbuster (SD 1.3/1.4/2); WildFake (SD 1.x/2.x); OpenFake (SD 1.5/2.1)	CRITICAL
w8 (2023–24)	SDXL 1.0, DALL-E 3, FLUX 1 Dev, FLUX 1 Schnell	Synthbuster, ImagiNet, Aeroblade	OpenFake (DALL-E 3, FLUX Dev/Schnell, SDXL); Manual-gen (DALL-E)	CRITICAL

Summary: Contamination confirmed in **7 of 9 windows** (all except w4 and partially w2).

Visual overlap diagram:

AI GenBench Window	Our Contaminating Data Sources	
w0 [GAN-era]	← WildFake(ProGAN, StyleGAN) + DRAGON→GenImage	HIGH
w1 [StyleGAN]	← WildFake(BigGAN, StyleGAN1) + DRAGON→GenImage	HIGH
w2 [DDPM]	← WildFake(StyleGAN2)	MEDIUM
w3 [ADM/VQGAN]	← DRAGON→GenImage (ADM, VQGAN)	HIGH

w4 [LaMa/Pal.]	← (minimal)		
			LOW
w5 [Glide/LD]	← DRAGON→Synthbuster(Glide,SD1.4)		
	+ DRAGON→GenImage(Glide)		HIGH
w6 [MidJourney]	← DRAGON→Synthbuster(MJv5)		
	+ OpenFake(MJv6/v7)		HIGH
w7 [SD1.x/2.x]	← DRAGON→Synthbuster(SD1.3/1.4/2)	■	
	+ WildFake(SD1.x/2.x)	CRITICAL ■	
	+ OpenFake(SD1.5/2.1)	■	
w8 [SDXL/DALLE]	← OpenFake(DALL-E 3, FLUX Dev/Schnell, SDXL)	■	
	+ Manual-gen(DALL-E,MJ)	CRITICAL ■	

4.4 Contamination Risk Assessment

Mechanism 1: DRAGON → Synthbuster (Direct)

DRAGON's documentation explicitly states it consolidates CiFAKE, Diffusion Forensics, Synthbuster, and GenImage into a unified dataset. Synthbuster is an AIGenBench source dataset covering SD 1.3/1.4/2, DALL-E 2/3, Adobe Firefly, Midjourney v5, and GLIDE.

Risk: Models trained on DRAGON have likely seen images from the **same Synthbuster collection** that populates AIGenBench windows w5–w8. Even if exact images differ (Synthbuster has ~9,000 images), the distribution is from the same generator checkpoints with the same generation settings.

Mechanism 2: DRAGON → GenImage (Direct)

GenImage is also both a DRAGON component and an AIGenBench source. GenImage provides ADM, BigGAN, GLIDE, VQGAN, VQDM, and SD variants — contributing to AIGenBench windows w3 and w5.

Mechanism 3: OpenFake → Windows w7, w8 (High Impact)

OpenFake contains SD 1.5/2.1 (w7), SDXL (w8), DALL-E 3 (w8), FLUX 1 Dev/Schnell (w8), and Midjourney v6/v7 (w6). Windows w7–w8 are the hardest evaluation steps in AIGenBench — any score on these windows using OpenFake-trained models is likely inflated.

Mechanism 4: WildFake → Windows w0–w3 (GAN-era)

WildFake includes ProGAN, StyleGAN 1/2, BigGAN, and ADM — all mapped to ForenSynths and GenImage in AIGenBench. The GAN-era windows (w0–w3) are considered "easier" by AIGenBench baselines, but contamination here still invalidates generalization claims for those steps.

Quantitative contamination estimate:

Assuming uniform generator coverage across our training sets, we estimate the generator distribution overlap $\$J\$$ per window:

Window	Overlap Coefficient $\bar{G}$	Interpretation
w0	~0.50	2 of 4 generators directly overlap
w1	~0.50	BigGAN + StyleGAN1 in WildFake
w2	~0.25	StyleGAN2 in WildFake
w3	~0.50	ADM + VQGAN in GenImage/DRAGON
w4	~0.00	No direct source overlap found
w5	~0.50	Glide in Synthbuster/GenImage
w6	~0.50	Midjourney v5 in Synthbuster
w7	~1.00	All 4 generators in our data (SD)
w8	~0.75	DALL-E 3, FLUX in OpenFake

4.5 Impact on Benchmark Validity

Submitting results to the AIGenBench leaderboard using models trained on our current data would produce **invalid generalization scores** because:

1. The "Next Period" evaluation is designed to measure generalization to **unseen** generators. If our training data includes those generators, the evaluation reduces to in-distribution performance.
2. The benchmark's temporal guarantees are voided: steps 6→w7 and 7→w8 (the critical Stable Diffusion and FLUX transitions) would show artificially high AUROC because those generators are in our training data.
3. The paper compares methods using AUROC averaged over all Next-Period steps, weighting each window equally. Contaminated windows (w7–w8) contribute ~25% of the aggregate score, meaning our contaminated submission would overestimate $\bar{G}$ by an estimated 5–15 AUROC points.

4.6 Clean Data Strategy

Table 6: Training Data Decision Matrix for AIGenBench Submission

Dataset	Decision	Justification
AIGenBench official train split	☐ Use	Benchmark-standard, no contamination by design
Open-Images-V7 (real images only)	☐ Use	Real images, no generator overlap
DRAGON	☐ Exclude entirely	Consolidates Synthbuster + GenImage, both AIGenBench sources
WildFake — GAN subset (ProGAN, StyleGAN, BigGAN)	☐ Exclude	Overlaps ForenSynths (w0–w1)

Dataset	Decision	Justification
WildFake — ADM / diffusion subset	❑ Exclude	Overlaps GenImage (w3)
WildFake — Midjourney v5	❑ Exclude	Overlaps Synthbuster (w6)
OpenFake — SD 1.4/1.5/2.1/XL	❑ Exclude	Overlaps ELSA-D3 / Synthbuster (w7-w8)
OpenFake — DALL-E 3, FLUX Dev/ Schnell	❑ Exclude	Directly in AIGenBench w8
OpenFake — Imagen, GPT Image 1, Ideogram, Grok-2	⚠ Audit	Not in AIGenBench sources; may be safe
Manual-gen-images — DALL-E, Midjourney	❑ Exclude	Overlaps w6/w8
Manual-gen-images — other generators	⚠ Audit	Verify generator identity before including

For internal (non-submission) evaluation: All current training data is acceptable.

Implementation: The safest approach for a clean AIGenBench submission is to **use only AIGenBench's own training split** (available via the GitHub repository) and add Open-Images-V7 real images as supplemental real-world examples.

5. Testing Strategy and Submission Process

5.1 Accessing AIGenBench Data

Repository: <https://github.com/MI-BioLab/AI-GenBench>

Setup:

```
git clone https://github.com/MI-BioLab/AI-GenBench.git
cd AI-GenBench
pip install -r requirements_train_and_evaluation.txt
pip install -r requirements_dataset_creation.txt
```

Training framework location: `training_and_evaluation/`

Key files:

```
training_and_evaluation/
├─ lightning_main.py      # Main training/evaluation entry point (PyTorch Lightning)
├─ run_training.sh        # Training shell script
├─ evaluate_all_windows.py # Runs evaluation across all 9 windows
├─ ai_gen_bench_metadata/ # Generator metadata, file ID lists
├─ algorithms/           # Model implementations (add ours here)
├─ dataset_loading/      # Dataset loaders for AIGenBench format
├─ lightning_data_modules/ # PyTorch Lightning DataModules
```

```

└─ training_configurations/ # YAML configs per model
└─ evaluation_scripts/     # Per-window evaluation utilities
└─ training_metrics/       # Metric tracking

```

Dataset files are NOT in the repository — they must be assembled from source datasets using scripts in `dataset_creation/`. The file ID lists (`resources/train_fake_file_ids.txt`, `train_real_file_ids.txt`, etc.) specify exactly which images to extract from each source.

5.2 Expected I/O Format and Training Protocol

Training input format:

```

# Image tensor
image: torch.Tensor # shape (B, 3, H, W), float32, range [0, 1]

# After normalization (ImageNet stats)
image = transforms.Normalize(
    mean=[0.485, 0.456, 0.406],
    std =[0.229, 0.224, 0.225]
)(image)

# Binary label
label: torch.Tensor # shape (B,), dtype int64, values {0: real, 1: fake}

# Optional generator class label (for multiclass training)
generator_label: torch.Tensor # shape (B,), dtype int64, values {0..35}

```

DetermAugment pipeline (applied at evaluation — fixed, cannot be customized):

```

transforms.Compose([
    # JPEG compression simulation
    transforms.Lambda(lambda x: jpeg_compress(x, quality=randint(50, 99))),
    # Gaussian blur
    transforms.GaussianBlur(kernel_size=5, sigma=uniform(0, 2)),
    # Additive Gaussian noise
    transforms.Lambda(lambda x: x + randn_like(x) * 0.01),
    # Resize
    transforms.Resize(int(224 * uniform(0.5, 2.0))),
    # Center crop to target size
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

```

CustomAugment (training — can be customized, but output volume is fixed by `am=4`): The `am=4` multiplier means each training image produces 4 augmented variants per epoch. Submitted methods may customize this augmentation, but the total number of gradient updates is held fixed.

Training protocol pseudocode:

```

model = YourModel()

for k in range(9): # 9 time steps
    # Load previous weights
    if k > 0:
        model.load_state_dict(torch.load(f"checkpoint_step_{k-1}.pt"))

    # Collect training data up to window k
    train_data = concat([window_data[j] for j in range(k + 1)])

    # Train for 1 epoch with augmentation multiplier 4
    for epoch in range(1):
        for batch in DataLoader(train_data):
            augmented_batches = [custom_augment(batch) for _ in range(4)]
            for aug_batch in augmented_batches:
                loss = criterion(model(aug_batch["image"]), aug_batch["label"])
                optimizer.zero_grad(); loss.backward(); optimizer.step()

    # Save checkpoint
    torch.save(model.state_dict(), f"checkpoint_step_{k}.pt")

    # Evaluate on next window (k+1) with DetermAugment
    if k < 8:
        next_period_auroc = evaluate(model, window_data[k + 1], determ_augment)
        print(f"Step {k}: Next Period AUROC = {next_period_auroc:.4f}")

```

5.3 Evaluation Metrics and Scoring

Per-step metrics reported:

Metric	Definition	Primary?
AUROC (Next Period)	AUROC on unseen w_{k+1}	□ Primary
AUROC (Past Period)	AUROC on $w_0, \dots, w_k$	Secondary
AUROC (Whole Period)	AUROC on all seen + next	Secondary
Accuracy (Next Period)	Accuracy at threshold 0.5 on w_{k+1}	Secondary

Aggregate metric for leaderboard:

$$\bar{G} = \frac{1}{8} \sum_{k=0}^7 \text{AUROC}(\text{Next}) \left(\theta_k, w_{k+1} \right)$$

(averaged over 8 Next-Period evaluations, steps 0–7)

Calibration note: Because DetermAugment applies random degradations, evaluations should be averaged over multiple runs with different random seeds to reduce variance. The benchmark paper uses 3 runs.

5.4 Submission Requirements

Per the benchmark website (<https://mi-biolab.github.io/aigenbench-website/>):

Requirement	Details
Public codebase	Full training + evaluation code must be publicly accessible (GitHub)
Public report	arXiv preprint or published paper; a detailed technical report suffices
Protocol compliance	Must follow the sliding window protocol with DetermAugment at evaluation
No leakage	Cannot use AIGenBench evaluation data for training
Foundation models	General-purpose pretrained models (CLIP, DINOv2, ImageNet) are permitted
No domain-specific pretraining	Models pretrained specifically for deepfake detection are not permitted

Methods that use the AIGenBench data but don't follow the protocol can still report results but **will not appear on the leaderboard**.

5.5 Adaptation Plan for DeepFakeDetector

Step 1: Environment setup

```
git clone https://github.com/MI-BioLab/AI-GenBench.git
cd AI-GenBench
pip install -r requirements_train_and_evaluation.txt
```

Step 2: Implement our model as an AIGenBench algorithm

Create `training_and_evaluation/algorithms/deepfake_detector_vit.py` following the repository's algorithm interface pattern. The model should:

- Accept `(B, 3, 224, 224)` input tensor
- Output binary logits or probabilities
- Support `model.train()` and `model.eval()` modes

Step 3: Configure training

Create `training_and_evaluation/training_configurations/deepfake_detector_config.yaml` specifying:

```
algorithm: deepfake_detector_vit
backbone: vit_large_patch14_dinov2
learning_rate: 5.0e-5
scheduler: cosine
warmup_steps: 500
augmentation_multiplier: 4 # am=4, fixed
batch_size: 32
```

Step 4: Run training across all windows

```
cd training_and_evaluation
bash run_training.sh --config training_configurations/deepfake_detector_config.yaml
```

Step 5: Evaluate across all windows

```
python evaluate_all_windows.py \
  --model_dir checkpoints/deepfake_detector_vit/ \
  --output_dir results/deepfake_detector_vit/
```

Step 6: Write technical report

A technical report summarizing our architecture, design choices, and results can be drafted based on this document and submitted to arXiv.

Step 7: Submit

Push code to a public GitHub repository and submit via the AIGenBench benchmark website with a link to the codebase and technical report.

6. Recommendations and Roadmap

6.1 Immediate Actions (Week 1)

1. **Close issue #48** — `research/fusion_model_design.md` already satisfies all criteria (completed in PR #50)
2. **Set up AIGenBench framework** locally: clone, install dependencies, verify dataset paths
3. **Run ViT-Base (current)** through AIGenBench's `evaluate_all_windows.py` with clean data to get an honest baseline $\bar{G}$
4. **Add JPEG compression augmentation** to all current training pipelines (independent of AIGenBench work)

6.2 Short-Term (Weeks 2-4)

1. **Fine-tune ViT-L/14 DINOv2** on AIGenBench's training split (clean) using the temporal protocol
2. **Ablation study:** Compare (a) ViT-L/14 DINOv2 alone vs (b) fusion with EfficientNet-B0 vs (c) fusion with FFT module
3. **Evaluate FFT module** on AIGenBench windows w7-w8 specifically — hypothesis: frequency artifacts in SDXL/FLUX may be detectable with log-magnitude FFT analysis

6.3 Model Architecture Upgrade Path

Current State	Target State
EfficientNet-B0 (5.3M, ImageNet)	EfficientNet-B0 (5.3M, ImageNet)
ViT-Base/16 (86M, ImageNet-21k)	→ ViT-L/14 (307M, DINOv2 self-supervised)
DeiT-Small (22M, ImageNet)	DeiT-Small (22M, ImageNet) [keep as alt.]
CompactFFNet (0.5M, scratch)	CompactFFNet (0.5M, scratch) [keep]
CompactGradientNet (1M, scratch)	CompactGradientNet (1M, scratch) [keep]
Fusion: 4-class LogReg	→ Fusion: ViT-L/14 + EfficientNet + FFT

6.4 Training Data Strategy Summary

Context	Recommended Training Data
AlGenBench leaderboard submission	AlGenBench train split + Open-Images-V7 (real) only
Internal evaluation (our test sets)	All current data (OpenFake, DRAGON, WildFake, etc.)
General product deployment	All current data + AlGenBench train split

6.5 Expected Performance Targets

Table 7: Projected $\bar{G}$ AUROC on AlGenBench

Configuration	Architecture	Training Data	Expected $\bar{G}$
Baseline (current)	ViT-Base/16	Clean split	~78-82%
Improved augmentation	ViT-Base/16 + JPEG aug	Clean split	~82-86%
Architecture upgrade	ViT-L/14 CLIP	Clean split	~90-92%
Primary target	ViT-L/14 DINOv2	Clean split	~93-95%
Ensemble (stretch)	ViT-L/14 DINOv2 + EfficientNet-B0 + FFT	Clean split	~94-96%

The ensemble target assumes FFT features provide complementary signal not captured by ViT-L/14's spatial attention — this is an open research hypothesis that should be validated experimentally.

7. References

1. **Pellegrini et al. (2025)** — "Al-GenBench: A New Ongoing Benchmark for AI-Generated Image Detection." Verimedia Workshop @ IJCNN 2025. [arXiv:2504.20865](https://arxiv.org/abs/2504.20865). DOI: [10.1109/IJCNN64981.2025.11228377](https://doi.org/10.1109/IJCNN64981.2025.11228377)

2. **Pellegrini et al. (2025)** — "Generalized Design Choices for Deepfake Detectors." Under review. [arXiv:2511.21507](#)
3. **AI-GenBench GitHub** — <https://github.com/MI-BioLab/AI-GenBench>
4. **Wang et al. (2020)** — "CNN-Generated Images are Surprisingly Easy to Spot... for Now." CVPR 2020. (CNNDetection / ForenSynths source paper)
5. **Dosovitskiy et al. (2021)** — "An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale." ICLR 2021. [arXiv:2010.11929](#)
6. **Oquab et al. (2024)** — "DINOv2: Learning Robust Visual Features without Supervision." TMLR 2024. [arXiv:2304.07193](#)
7. **Radford et al. (2021)** — "Learning Transferable Visual Models From Natural Language Supervision (CLIP)." ICML 2021. [arXiv:2103.00020](#)
8. **Frank et al. (2020)** — "Leveraging Frequency Analysis for Deep Fake Image Recognition." ICML 2020. [PDF](#)
9. **Caron et al. (2021)** — "Emerging Properties in Self-Supervised Vision Transformers (DINO)." ICCV 2021. [arXiv:2104.14294](#)
10. **Zhu et al. (2017)** — "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks (CycleGAN)." ICCV 2017.
11. **Karras et al. (2019/2020)** — "A Style-Based Generator Architecture for Generative Adversarial Networks (StyleGAN 1/2)." CVPR 2019/2020.
12. **Ho et al. (2020)** — "Denoising Diffusion Probabilistic Models (DDPM)." NeurIPS 2020. [arXiv:2006.11239](#)
13. **Rombach et al. (2022)** — "High-Resolution Image Synthesis with Latent Diffusion Models (Stable Diffusion)." CVPR 2022. [arXiv:2112.10752](#)

This document was produced as part of the McMaster AI Society DeepFakeDetector project in response to [issue #65](#).